UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ



LUCRARE DE LICENȚĂ

Simularea interacțiunilor pradă-prădător prin NEAT

propusă de

Daniel Apostu

Sesiunea: iunie/iulie, 2023

Coordonator științific

Lect. Dr. Croitoru Eugen

UNIVERSITATEA ”ALEXANDRU-IOAN CUZA” DIN IAȘI

FACULTATEA DE INFORMATICĂ

Simularea interacțiunilor pradă-prădător prin NEAT

Daniel Apostu

Sesiunea: iunie/iulie, 2023

Coordonator științific

Lect. Dr. Croitoru Eugen

Cuprins

Introducere **2**

Motivație **5**

1 NEAT **7**

1.1 Retele Neuronale Artificiale 8

1.2 Problemele in TWEANNs si inovatiile in NEAT 9

1.2.1 Codificare 9

1.2.2 Initializarea populatiei 10

1.2.3 Problema conventiilor concurente si numerele de inovatie 11

1.2.4 Operatori Genetici 11

1.2.5 Speciație 13

1.2.6 Reproducerea 14

2 rtNEAT **15**

2.1 Inlocuirea agentului cel mai putin adaptat 15

2.2 Generarea copilului 16

2.3 Parametrii de rulare a algoritmului 17

3 Mediul Virtual **18**

3.1 Vizualizarea principala 18

3.2 Mâncarea 19

3.3 Creaturile 20

3.4 Managementul Algoritmului 25

4 Detalii de implementare **27**

4.1 Genomul in NEAT 27

4.2 Retelele neuronale din NEAT 29

4.3 Algoritmul NEAT 35

5 Setari experimentale **40**

6 Rezultate **41**

Concluzii **42**

Introducere

În ciuda faptului că dinamica populației a atras din punct de vedere istoric interesul multor oameni de știință, suntem încă departe de a înțelege cum și de ce populațiile fluctuează în ecosistemele naturale. Motivul principal este că populațiile sunt sisteme complexe, cu proprietăți emergente care sunt imposibil de determinat prin suma părților lor.[Oro13]
Intr-un mediu virtual, am putea izola si controla variabilele din mediu pentru a studia anumiti factori specifici care influenteaza aceasta dinamica. Specific, vom simula si studia interactiunile prada-pradator, impactul lor reflectându-se in dinamica populatiei in urmatoarele moduri:

- **Reglarea populatiei** - prădarea reprezinta un mecanism regulator atat pentru populatia prazii cat si a pradatorilor, ce previne prada din a ajunge la densitati prea mari, iar densitatea prădătorilor e controlata de cea a prazii.

- **Dinamica ciclica a populatiei** - interactiunile prada-pradator pot rezulta in ciclicitate a populatiilor, unde numarul prazii si a pradatorilor pot sa prezinte fluctuatii in timp in mod regulat. In studierea impactului pe care l-ar avea incalzirea globala asupra dinamicii sistemelor ecologice sa observat chiar ca durata acestor cicluri se micsoreaza trecand de la temperaturi calde la cele mai reci[DeL20].

- **Dinamici evolutive** - presiunea impusa de mecanismul de pradare poate duce la adaptari evolutive. In mediul natural prăzile pot dezvolta mecanisme defensive(coarne, camuflaj, viteza) iar pradatorii deasemenea pot suferi schimbari fizice dar si in strategiile de vanatoare.

În aceasta lucrare vom dezvolta un mediu virtual cu o populație de prada și una de prădători, cu scopul de a analiza transformarile suferite in populatii dupa criteriile de mai sus. Mediul va fi dezvoltat in motorul de jocuri Unity, si il vom descrie in capitolul 3.El va include 4 pereti pozitionati pentru a crea o arie inchisa. In cadrul acelei

arii vor fi generati in mod constant peleti de energie care vor fi consumati de prada, va fi generata populatia de prada si populatia de pradatori. Evolutia organismelor din fiecare populatie se va preta pe dezvoltarea comportamentelor, nu a trasaturilor fizice. Pentru a directiona aceasta evolutie, vom folosi un algoritm din clasa algoritmilor evolutivi.

De-a lungul dezvoltarii familiei de algoritmi evolutivi, apar anumite etape cheie in care se dezvolta ramuri noi. Enumeram cateva din ramuri in urmatorul mod:

- **Algoritmii Genetici**(GA) - arie de studiu în care pionier este John Holland prin cartea sa "Adaptation in Natural and Artificial Systems", publicată în 1975[Hol92]. In ea el introduce utilizarea unei populatii de indivizi, a operatorilor genetici precum mutatie si crossover, si a mecanismelor de selectie inspirate de selectia naturala.

- **Strategii Evolutive**(ES) - dezvoltarea acestei ramuri poate fi atribuita lui Rechenberg și Hans-Paul Schwefel care sau ocupat de studiul ei în anii '60 și '70. In 1973 Rechenberg publica cartea "Evolutionsstrategie"[Rec73], in care documenteaza cercetarea sa pe probleme de optimizare continua.

- **NeuroEvolutie** - aceasta clasa de algoritmi cauta prin spatiul de ponderi si structuri o retea neuronala cu performante bune pentru a anumita sarcina. Practic algoritmii de neuroevolutie permit gasirea unui comportament care faciliteaza rezolvarea problemelor de **reinforcement learning** complexe[MM95]. De obicei, in cadrul algoritmilor NE, se alege o tipologie(structura de noduri si conexiuni) pentru retele ce urmeaza sa treaca prin procesul de evolutie, iar schimbarile au loc doar pe ponderile conexiunilor. **NEAT** este o abordare inedita a neuroevolutiei pe parcursul careia se modifica atat tipologia retelei cat si ponderile conexiunilor. Astfel este scutit efortul uman de a crea designul unei retele. La inceptia acestei metode a stat Kenneth O. Stanley, cu articolul s-au din 2002 "Evolving Neural Networks through Augmenting Topologies"[SM02]

In cadrul acestei teze, comportamentele fiecarui individ din populatie, adica actiunile pe care le fac la fiecare pas de timp vor fi dictate de cate o retea neuronala. Ca algoritmi de neuroevolutie vom folosi **NEAT** si **rtNEAT**. Acesti algoritmi vor fi descrisi fiecare in capitolele 1 si 2.In cadrul unui experiment in care vom include doar speciile de prada, in care excludem analiza populatiei ca numar de indivizi, vom identifica care

dintre algoritmi determina o progresie mai rapida in materie de acumulare de energie in medie si cata energie acumuleaza cel mai bun membru al populatiei. Un experiment similar a fost descris in lucrarea lui T. Gelles si M. Sanchez[Gel16] in care incearca sa evolueze comportamentul furnicilor de a plasa feromoni ca un mod de a comunica, si demonstreaza ca **rtNEAT** are mai mult succes in a evolua acest comportament decat **NEAT**.

Pentru alte experimente in materie de interactiune prada-pradator putem face referinta la lucrarea "An evolutionary approach for interactive computer games"[YLH04]. In cadrul ei autorii proiecteaza un joc numit "Dead End", in care jucatorul are scopul de a ajunge la iesirea din partea opusa a ecranului de unde este el generat, iar o echipa de 8 "Dogs", ai caror miscari sunt dictate de cate o retea neuronala, au scopul de al prinde inainte ca acesta sa ajunga la iesire, deci trebuiau sa dezvolte comportamente de urmarire si de cooperare. Pentru evolutia cainilor au fost folositi 2 algoritmi de invatare proprii, unul offline si unul online. Ambii aduceau schimbări doar ponderilor conexiunilor din retea, nu si a topologiei retelelor. Ei au identificat ca agentii antrenati in modul online prezinta o adapabilitate mare la schimbarea strategiilor de joc a jucatorilor, deci demonstreaza eficacitatea metodei impotriva jucatorilor umani.

Datorita concluziilor acestor 2 lucrari, vom sustine ipoteza ca **rtNEAT**, care este un algoritm de invatare online, va determina o evolutie mai rapida a indivizilor, in termeni de energie acumulata. Deasemenea, doar**rtNEAT** ne permite sa facem o analiza a populatiilor pe timp indelungat, deoarece **NEAT** reseteaza populatile la fiecare generatie. Daca identificam ca **NEAT**, care este un algoritm offline, produce o invatare mai rapida, il vom folosi pentru a evolua comportamentele indivizilor, iar retelele antrenate astfel le vom integra in **rtNEAT** pentru a face analiza dinamicii populatiei. In capitolul 4 vom detalia implementarea mediului, a algoritmilor, a retelelor, si a senzorilor folositi. In capitolul 5 vom prezenta experimentele pe care vrem sa le conducem si setarile pentru algoritmi in cadrul experimentelor. In final, 6 prezentam rezultatele si anlizam care sunt factorii care le-au influentat.

Motivație

Pe parcursul dezvoltării discipline informaticii, natura a servit ca inspirație pentru oamenii de știința care se ocupau cu dezvoltarea de noi algoritmi. Astfel a aparut clasa de **algoritmi evolutivi**, din care se branșează mai departe alte familii de algoritmi: **algoritmii genetici**, **strategii evolutive**, **programarea genetica** și **algoritmii de neuro-evoluție**.

Aceasta familie de algoritmi a ajutat la dezvoltarea multor solutii unde algoritmii clasici deterministi sunt prea costisitori sau ineficienti. Un exemplu ar fi o antena X-band evoluata pentru misiunea NASA din 2006 numita Space Technology 5, al carei design a fost gasit prin intermediul unui algoritm evolutiv[Hor+06].

Pe parcursul timpului, studiul in natura a intercatiunilor dintre prada si pradator a permis extragerea de informatii valoroase despre dinamica ecosistemelor. Datorita avansului rapid al inteligentii artificiale si a algoritmilor evolutivi, avem oportunitatea de a cerceta aceste intercatiuni intr-un mediu virtual.

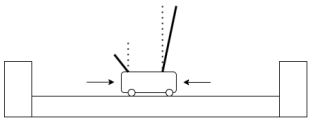
Aceasta teza de licenta propune utilizare algoritmilor **NEAT**(NeuroEvolution of Augmenting Topologies) si **rtNEAT**(real time NEAT), plus mediul de dezvoltare de jocuri **Unity**(2D) pentru crearea unei simulari dinamice a intercatiunilor dintre speciile de prada si cele pradator. Intelegerea acestor intercatiuni este esentiala pentru a descoperi factorii care influențează dinamica populațiilor, coexistența speciilor și stabilitatea ecosistemelor. Utilizand un mediu virtual, putem studia interactiunile prada-pradator in izolatie, putem controla variabilele de mediu, si putem repeta experimente de mai multe ori intr-un mod consistent(relativ la variabilitatea naturii). Algoritmul NEAT, datorita capacitatii sale de a evolua atat ponderile cat si topologiile rețelelor neuronale, reprezinta un motor puternic pentru modelarea comportamentelor și proceselor de luare a deciziilor ale prădătorilor și prăzii, cu scopul de a supravetui mai mult. Prin evoluția rețelelor neurale ale prăzii, putem explora comportamentele de căutare a hranei, tactici de evitare și strategii de supraviețuire, iar pradatorii ar trebui sa dezvolte

strategii de vanatoare, cu adaptari pentru schimbarile din mediu.
Motorul de joc Unity permite sa vizualizam in timp real comportamentele pe care le dezvolta orgnismele, sa monitorizam noile specii care apar din ambele categorii de organism, sa adaugam potential noi obstacole, noi senzori, sau noi abilitati prazii si pradatorilor, si sa controlam variabilele si viteza simularii.
În concluzie, această teză de licență își propune să îmbine domeniile inteligenței artificiale, al evolutiei biologice si al ecologiei, utilizând algoritmul NEAT și Unity pentru a simula interacțiunile dintre pradă și prădători. Prin studierea acestor dinamici ne propunem sa contribuim intelegerea sistemelor ecologice, cat și la educare populatiei pe larg asupra acestor interactiuni prin intermediul vizualizarilor. Ipoteza noastra e ca studiind evolutia populatiilor prazii si pradatorilo aceste se vor conforma ecuatiilor Lotka-Voltera.

Capitolul 1

NEAT

Algoritmul NEAT a fost descris pentru prima oara de Keneth O. Stanley si Risto Miikkulainen in articolul lor ”Evolving Neural Networks through Augmenting Topologies”[SM02]. In abordarile clasice ale **NeuroEvolutiei**(NE), se alege o topologie initiala a retelelor neuronale iar in timpul experimentelor algoritmii cauta doar in spatiul ponderilor conexiunilor. Ins topologia(structura) retelelor area deasemenea un impact asupra comportamentelor pe care le formeaza o retea. Un argument bun pentru a aplica evolutia atat asupra ponderilor cat si topologiei vine din partea lui Gruau[Gru93], care sustinea ca evolutia structurii unei retele economiseste timpul pe care l-ar fi folosit un om in a decide topologia pentru o anumita problema de NE. A demonstrat asta evoluând topologia si conexiunile unei retele care a reusit sa rezolve cel mai dificil benchmark de pole balancing de atunci(1996), double pole balancing.



Totusi, aceasi problema a fost rezolvat de 5 ori mai rapid de **Enforced Subpopulations**(ESP)[GM99], astfel, autorii NEAT si-au propus sa demonstreze un argument mai puternic pentru evolutia structurilor, anume ca modificand atat topologia cat si ponderile putem spori cu mult performanta algoritmilor NE. NEAT poate face asta datorita faptului ca minimizeaza structurile retelelor pe tot parcursul algoritmului. Au existat pe parcursul timpului mai multe metode de evolutie atat pentru topologiile cat si ponderile retelelor neuronale, ale caror ideii autorii le-au unit in [SM02] sub termenul de **”TWEANS”**(Topology and Weight Evolving Artificial Neural Networks). Pentru dezvoltarea NEAT Kenneth O. Stanley a adresat un numar de probleme nerezolvat la clasa de algoritmi TWEANS. Inainte de asta ar trebui totusi sa prezentam cum functioneaza retelele neuronale si modul lor de a invata.

1.1 Retele Neuronale Artificiale

In natura, o retea neuronala reprezinta o retea de neuroni interconectati, care sunt responsabili de functionarea sistemului nervos. Sunt specializati in transmiterea informatiei prin semnale electrice si au 4 componente principale, dendritele, corpul celular, axonul si butonii terminali.



Fiecare neuron primeste informatie prin dendrite si le trasmite prin axon printr-un semnal electric cu toata puterea sa, daca a primit destula informatie prin Dendrite. Neuronii sunt conectati intre ei prin sinapse, iar sinapsele isi pot adapta numarul de receptori pentru a controla puterea semnalului care vine de la un neruon. O retea neuronala artificiala este compusa dintr-o serie de noduri conectate intre ele ca un graf. Ca structura, o retea are un strat de noduri de iesire, unul de noduri de iesire, iar retelele deep au si un numar de straturi ascunse. Fiecare conexiune are o pondere asociata ei ce reprezinta puterea conexiunii. Aceste ponderi se pot adapta astfe incat reteau sa functioneze conform asteptarilor. Conform Teoriei Aproximarii Universale, o retea neuronala ar putea sa ajunga aproape de a a aproxima orice functie pentru care este conceputa. Spre deosebire de creier, in o retea neuronala pot exista mai multe tipuri de neuroni, determinati de o functie de activare. Daca nu folosim o functie de activare, valoarea unui neuron este reprezentata de o suma ponderata a intrarilor plus optional un bias al neuronului.



Daca este folosita o functie de activare, atunci semnalul nodului va fi outputul functiei de activare, iar input-ul este suma ponderata dintre conexiunie si intrarile nodului.

Retelele neuronale artificiale traditionale sunt feedforward, adica datele sunt procesate intr-o singura directie, de la input la output, iar invatarea se face prin algoritmul de backpropagation. In documentul care descrie NEAT, nu sunt expuse restrictii asu-

pra conexiunilor care se pot forma, deci algoritmul poate duce la generarea retelelor neuronale recurente.



Acestea pot procesa date secventiale, deoarece contin conexiuni ciclice, ce se pot forma intre oricare noduri, inclusiv nodul de la care conexiunea a pornit. La fiecare pas de timp, in RNN nodurile proceseaza atat intrarea curenta cat si iesirea nodului recurent anterior, ceea ce permite retelei sa utilizeze informatii din trecut. Pentru antrenare NEAT nu foloseste backtracking, ci operatorii genetici.

1.2 Problemele in TWEANNs si inovatiile in NEAT

1.2.1 Codificare

O problema pe care trebuie sa o rezolve orice TWEANN este modul in care specificam genomul unui individ. Aici sunt doua categorii mai mari, **codificare indirecta** si **directa**. Codificarea indirecte de obicei doar specifica reguli pentru crearea fenotipului, si sunt specifice algoritmilor care le folosesc Pentru codificarea directa optiuni sunt **codificarea binara**, care foloseste codificare clasica a grafurilor cu siruri de biti, precum "Structured Genetic Algorithm"(sGA)[DM92], care reprezinta prin siruri de biti matriciea de conexiuni a retelei, insa cele mai intalinte codificari sunt cele de tip graf, care reprezinta grafurile in mod mai explicit. NEAT foloseste o codificare directa, din cauza ca codificarile indirecte neavand un maping direct la fenotip, daca nu suntem foarte familiari cu procesele genetice, cautarea poate fi perturbata in moduri inpredicitibile si greu de detectat. Codificare pe care o alegem este foarte importanta, deoarece aceasta afecteaza costul operatiei de crossover, astfel ca exista TWEANNs "nonmating", care nici macar nu folosesc acest operator

Codificarea NEAT a fost proiectata cu scopul de a permite alinierea genom-urilor pen-

tru crossover fara a fi nevoie de o analiza costisitoare a topologiei. Aceasta codificare este reprezentata de o lista de node genes, si o lista de connection genes. Pentru fiecare node gene este specificat id-ul sau si tipul de nod, care clasifica nodurile in noduri de intrare, de iesire, si noduri ascunse. Primele sunt nodurile de intrare, dupa care cele de iesire iar cele hidden sa adauga la capatul listei in urma mutatiilor. Fiecare conection gene contine: un in-node si un out-node, care exprima nodurile si directia conexiunii, ponderea asociata cu conexiunea, o valoare boolean care indica daca conexiunea ar trebui sa fie exprimata in fenotip, si un numar de inovatie(prezentam de ce este necesar in sectiunea 1.2.3), care permite gasire geneleor care corespund din diferiti indivizi si e una din piesele centrale ale algoritmului.



Figura 1.1: Genotipul și fenotipul exprimat de acesta

1.2.2 Initializarea populatiei

La majoritatea algoritmilor din clasa TWEANN, populatia initiala este generata cu topologii random, pentru a asigura diversitate in structurile retelelor. Totusi, initializare in modul asta produce multe probleme pentru TWEANNS:

1. Nu avem nici o garantie ca informatia poate ajunge din fiecare intrare la fiecare iesire, ceea ce poate face ca unele intrari sa fie irelevante. Timp important de procesare va fi pierdut pentru a scapa de astfel de retele.

2. Ideal am dori ca solutiile obtinute sa fie minime, adica un numar minim de noduri si conexiuni, deoarece o retea mai complexa necesita mai mult timp pentru optimizarea ponderilor. Cu initilizare random, o mare parte din timp ar fi folosita pentru filtrarea structurilor extra care nu ar fi trebuit sa existe de la început. Evaluarea relelor mai complexe ia mai mult timp, dar prin aceasta initializare nu avem nici o garantie ca solutia va fi minima, deoarece orice retea atata timp cat are un fitness mare poate domina peste celelalte.

Solutia NEAT este sa inceapa in mod minim. Populatia initiala are fiecare nod de intrare conectat cu cele de iesire cu ponderi random, si nici un nod ascuns, astfel solutia va adauga structuri noi doar atunci cand aceste ajuta la imbunatatirea fitness-ului. Astfel se asigura minimalitatea nu doar la final dar pe parcursul tuturor generatiilor. Ar trebui sa precizam ca initial, adaugand unei retele structuri noi, care nu au fost optimizate, va face ca fitness-ul sa scada initial, deci indivizii cu inovatii topologice sunt la risc sa dispara, asta fiind un alt motiv pentru care majoritatea algoritmilor TWEANN folosesc initializare cu topologii random. Pentru a evita pierderea inovatiilor, NEAT foloseste **"Speciatiea"**, pe care o vom detalia in sectiunea 1.2.5.

1.2.3 Problema conventiilor concurente si numerele de inovatie

Problema conventiilor concurente apare atunci cand exista mai mult de un singur mod de a reprezenta o retea neuronala optimizata pentru o problema. Crossover in cazul a doua astfel de solutii poate duce la pierderea de informatie in cadrul noului copil format. Aceasta problema este inca mai grava in cadrul TWEANNs, deoarce solutii pentru o problema pot fi reprezentate prin topologii complet diferite, iar genomul fiecăreia poate avea lungimi diferite, agravand problema conventiilor concurente. Solutia in cadrul NEAT la aceasta problema este inspirata din solutia naturii in cadrul reproducerii sexuale, adica **omologia**, doua gene fiind omoloage daca ele sunt trasaturi ale aceleiasi alele. NEAT stabileste omologia dintre gene prin intermediul **marcatorilor istorici**, reprezentati de **numere de inovatie**, care permit stabilirea originii istorice a unei gene, iar astfel putem identifica si genele omoloage. Vom presznta cum apar si cum sunt folosite numerele de inovatie in sectiunile 1.2.4 si 1.2.5

1.2.4 Operatori Genetici

NEAT foloseste 2 operatori genetici pentru modificarea genomul-ui indivizilor, mutatia (fig 1.2) si crossover. Prin mutatie, ca si la ceilalti algoritmi de NE, se pot modifica ponderile, prin perturbarea lor sau prin asignarea unei noi valori random, insa pentru NEAT mutatie este si modalitate prin care introducem schimbari structurale. Sunt folosite 2 tipuri de mutatie structurala, "add connection", prin care se adauga o conexiune cu pondere random intre oricare 2 noduri care nu erau conectate deja, si "add node", care alege o conexiune in mod aleatoriu, si adauga un nou nod in locul ei. Mai exact, conexiunea aleasa este dezactivata, si se creaza o noua conexiune intre

nodul de intrare al conexiunii dezactivate si nodul nou creat, cu ponderea de 1, iar conexiunea dintre nodul nou si nodul de iesire primeste ponderea celei dezactivate. Astfel se minimizeaza efectul initial al modificarilor structurale, iar ponderile noilor conexiuni vor fi modificate in generatiile viitoare. De fiecare data cand producem o



Figura 1.2: Cele 2 tipuri de mutatie, ”add connection” si ”add node”

modificare structurala, fiecare gena de conexiune are adaugat si un nou numar de inovatie din punct de vedere global. Acest numar ne permite sa stabilim originea istorica a acestei gene. Daca in cursul unei generatii, pastram o lista a inovatiilor care au avut loc iar in genomul a 2 indivizi diferiti va aparea o aceasi gena de conexiune, poate doar cu ponderi diferite, aceste gene vor primi ambele acelasi numar de inovatie, pentru ca reprezinta aceasi structura dar care a aparut in mod indepedent in cadrul fiecareai. Crossover reprezinta incrucisarea genomului celor 2 parinti, pentru a crea un copil ce are structuri din partea ambilor Acum, datorita marcatorilor istorici, in cadrul crossover doua genomuri pot fi aliniate dupa genele care au numere de inovatie egale, acestea sunt gene ”matching”. Genele care nu sunt matching sunt fie ”disjoint” daca sunt in limita numerelor de inovatie a ambilor parinti, fie ”excess” daca unul din parinti are numere de inovatii mai mari decat cel mai mare numar al celuilalt parinte. In cadrul



Figura 1.3: Procesul de ”matching” pentru doua genomuri diferite

compunerii copilului, acesta mosteneste in mod aleator genele matching ale ambilor parinti, insa doar genele disjoint si excess ale parintelui cu cel mai mare ”fitness”. In

cazul in care ambii parinti au fitness egal, atunci si genele excess si disjoint sunt mostenite aleator. Pentru genele dezactivate, este o probabilitate prestabilita ca o gena sa ramana dezactivata daca ea e asa in cadrul oricaruia dintre parinti.
Prin intermediul acestor 2 operatori genetici, este introdusa diversitate in cadrul populatiei. Totusi, de obicei, datorita faptului ca strutucturile mai mici au mai putin conexiuni, aceste se pot optimiza mai rapid, iar adaugarea de noi structuri, pana acestea sunt optimizate, va scadea din fitness-ul retelei, deci inevitabil noile inovatii ar disparea rapid din o populatie normale, cu tote ca potential ele ar fi chiar necesare. Din acest motiv in NEAT este nevoie de **Speciatie**, pe care o prezentam in sectiunea 1.2.5.

1.2.5 Speciație

Speciatia este un mod de a proteja noile inovatii topologice aparute. Ideea este de a permite indivizilor sa concureze in propria lor nisa, nu cu intreaga populatie. Pentru a detalia, topologiile similare din cadrul populatiei vor fi incadrate in specii, organismele din aceleasi specii concureaza in principal intre ele, si astfel au timp sa isi optimizeze conexiunile. Cum anume sunt determinate speciile si organismele din cadrul lor? Tot cu ajutorul numerelor de inovatie. Numarul de gene ”desjoint” si ”excess” sunt un indicator bun al distantei din punct de vedere al compatibilitatii intre 2 genoame. Astfel, autorii NEAT au ajuns la o formula simpla care sa masoare compatibilitatea ca o distanta intre 2 structuri diferite in NEAT(notam δ) in termeni de numar de gene excess E si numar de gene desjoint D, plus media diferentei dintre genele matching $\overline{W}$:

$$\delta = \frac{c1 * E}{N} + \frac{c2 * D}{N} + c3 * \overline{W} \tag{1.1}$$

Coeficientii $c1$, $c2$ si $c3$ ajusteaza importanta acestor factori, iar N, care reprezinta numarul de connection genes a genomului cel mai lung, normalizeaza valorile conform dimensiunii genomului. Prin calculul valorii $delta$ putem desfasura procesul de speciatie cu ajutorul unei valori stabilite in prealabil, pragul de compatibilitate δt. Procesul de speciatie se desfasoara in urmatorul mod. Fiecare specie are un genom care o reprezinta. Fie un genom g din generatia curenta, acesta este plasat in prima specie cu a carui reprezentant este compatibil, adica $\delta < deltat$. Daca nu exista nici o specie compatibila, se formeaza una noua cu g ca reprezentat al acesteia.

1.2.6 Reproducerea

NEAT foloseste un mecanism numit **"explicit fitness sharing"** pentru a evita ca o specie sa devina prea mare si sa ajunga de dimensiunea intregii populatii. Practic, toate organismele din aceasi specie impartasec fitness-ul speciei lor. Fitness-ul ajustat al organismului i este calculat prin formula:

$$f\prime_i = \frac{f_i}{\sum_{j=1}^{n} sh(\delta(i,j))} \tag{1.2}$$

unde $f\prime_i$ e fitness-ul ajustat al individului i, n e numarul de organimse din populatie, iar functia sh are valoare 0 cand $\delta(i,j)$ este peste δ_t, dar 1 altfel. Astfel, partea de jos a fractiei reprezinta doar numarul de indivizi din aceasi specie cu organismul i. Fiecarei specii ii este acordat un numar diferit de membri potentiali depinzand de suma a $f\prime_i$ a membrilor speciei dupa formula:

$$n_k = \frac{\overline{F_k}}{\overline{F_{tot}}}|P| \tag{1.3}$$

unde k reprezinta o specie, n_k e nr de membri asignati speciei, $|P|$ este numarul de indivizi in populatie, $\overline{F_k}$ este media fitnessului speciei, adica $\overline{F_k} = \sum_i f\prime_i$, iar $\overline{F_{tot}} = \sum_k \overline{F_k}$ Intreaga populatie este apoi inlocuita de copii membrilor mai buni ai fiecarei specii, membrii mai slabi fiind eliminati in prealabil. Scopul final al speciatiei este protejare inovatiilor topologice, iar sistemul intreg cauta o solutie in cel mai rapid mod posibil, deoarece inovatii noi apar si sunt pastrate doar atunci cand e necesar.

Capitolul 2

rtNEAT

rtNEAT este o metoda bazata pe NEAT(vedeti capitolul 1), dezvolata de aceasi autori plus Bobby D. Bryant[SBM05]. Acesta este o adaptare a NEAT, gandit pentru jocuri, pentru evolutia retelelor neuronale in real-time, permitand agentilor controlati de ele sa se imbunatateasca in timpul desfasurarii jocului. Penru a demonstra acest concept de joc, a fost dezvoltat "NERO"(Neuro-Evolving Robotic Operatives)[Uni03], un joc bazat pe rtNEAT, in care un jucator antreneaza o echipa de roboti virtuali pentru a se lupta cu robotii altei echipe. Una dintre principalele observatii care au permist dezvoltarea rtNEAT este faptul ca intrun joc toata populatia joaca in acelasi timp, deci fitness-ul este inregistrat constant, astfel agentii tot sar putea adapta constant. Am considerat acest algoritm potrivit deoarece in mediul nostru virtual, ne dorim ca organismele sa evolueze si sa se adapteze in real-time, pe baza raspunsurilor pe care le primesc de la diversele interactiuni cu mediul, astfel am putea vedea o evolutie in termeni de comportamente cat si ca numar de indivizi, deoarece algoritmul NEAT reseteaza populatia la fiecare generatie.
In cadrul rtNEAT, la fiecare n intervale de timp, se executa o serie de operatie pentru inlocui un organism cu performante scazute cu copilul dintre 2 organisme mai performante, alese in mod probabilist astfel incat sa prezerve speciatia din NEAT.

2.1 Inlocuirea agentului cel mai putin adaptat

Prin aceasta operatie, se inlocuieste un agent putin performant cu unul potential mai performant, dar in asa fel incat se pastreaza dinamica de speciatie din NEAT. Pentru asta, alegem sa eliminam agentul cu cel mai mic "fitness ajustat", deoarece daca



Figura 2.1: Ciclul de inlocuiri in cadrul rtNEAT

alegerea sar face dupa cel mai rau fitness "neajustat", ar fi eliminate intii organismele din specii in care a aparut o inovatie topologica noua si care nu a avut timp sa se optimizez, astfel speciatia nu ar mai proteja inovatia. Fitness-ul ajustat este calculat la fel ca in cadrul NEAT, adica pentru individul i, $f\prime_i = \frac{f_i}{|S|}$, $|S|$ fiind numarul de indivizi din aceasi specie cu i.

Trebuie deasemenea sa ne asiguram ca fiecare individ a supravetuit destul inainte sa fie eliminat, deoarece altfel ar fi eliminati mereu cei mai noi indivizi, care nu au avut timp sa acumuleze fitness. Din acest motiv, rtNEAT elimina doar agentii care au supravetuit minimul de timp m. Acest parametru e setat experimental. Dupa eliminarea unui agent, este important sa estimam din nou fitnessul mediu $\tilde{F}$, deoarce acesta se schimba, si este folosit in urmatorul pas al algoritmului.

2.2 Generarea copilului

Din cauza ca nu generam din nou intreaga populatie la inceputul fiecarei populatii, nu poate fi utilizata formula 1.3, dar dinamica speciatie poate fi conservata daca alegem parintele in mod probabilist:

$$Pr(S_k) = \frac{\overline{F_k}}{\overline{F_{tot}}} \tag{2.1}$$

adica probabilitate de a alege o specie anume pentru parinti este proportionala cu raportul dintre fitness-ul mediu al speciei si suma fitness-urilor medii pentru toate speciile. Noul copil este format prin crossover dintre 2 indivizi din specia aleasa plus mutatie.

Daca folosim un prag de compatibilitate dinamic δ_t, atunci nu putem face la fel ca in "NEAT", adica sa il actualizam inainte de crearea urmatoarei generatii, deoarece aproape toata populatia va ramane in aceasi specie. In rtNEAT, este ales un numar de

inlocuiri de indivizi intre care se fac ajustari ale δ_t, de exemplu la fiecare 3 inlocuiri, iar pe baza noului δ_t fiecare individ din populatie va fi repartizata din nou la una speciile existente.

2.3 Parametrii de rulare a algoritmului

Toti pasii algoritmului rtNEAT, pe care iam discutat mai sus, trebuie executati la intervale regulate, adica la fiecare n ticuri de timp. Parametrul n trebuie ales astfel incat rata de inlocuiere sa le permita agentilor sa supravetuiasca destul cat sa ajunga la timpul minim m, pentru a putea fi evaluati, adica sa treacat din statutul de neelegibili pentru inlocuire in eligibili, dar si destul de des cat sa nu se incetineasca evolutia la un grad frustrant. Pentru a determina parametrii potriviti, autorii au stabilit o lege a eligibilitatii, care stabileste ce proportie din populatie ne putem astept sa fie nelegibila odata ce algoritmul a trecut de primele generatii cand toata populatia este nelegibila:

$$I = \frac{m}{|P|n} \tag{2.2}$$

unde I e fractia nelegibila din populatie, m e durata minima de viata pe care o poate avea un individ, n este numarul de ticuri intre procesele de inlocuire, iar $|P|$ numarul de indivizi in populatie. Practic, cu cat este mai mare populatia si numarul de ticuri intre inlocuiri, cu atat este mai greu este inlocuita populatia in intregime, deci mai putini agenti neelegibili, iar cu cat m este mai mare, cu atat mai multi vor fi sub pragul eligibilitatii. Astfel, putem construi o formula pentru n:

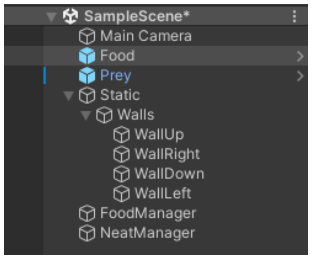
$$n = \frac{m}{|P|I} \tag{2.3}$$

Astfel, parametri pe care trebuie sa ii alegem noi raman doar m, $|P|$ si I.
Procesele descrise pana acum permit rtNEAT sa replice dinamica NEAT, dar in real-time.

Capitolul 3

Mediul Virtual

In acest capitol vom descrie mediul Unity in care se desfasoara simulara. O scena in Unity este compusa din o multime de **GameObjects**, care interactioneaza intre ele in timpul jocului, sau care prezinta doar aspecte estetice. Mai departe vom prezenta obiectele din cadrul simularii, proprietatile lor, si cum interactioneaza cu celelalte obiecte din mediu. Structura scenei a fost inspirata din proiectul "Simulated Fish Ecosystem"[Zol21].



3.1 Vizualizarea principala

In mod implicit, inainte sa dam start oricarei simulari, in mediu avem 2 structuri principale: camera(**Main Camera**) si peretii(**Walls**). **Main Camera** stabileste dimensiunea **Game view**-ului (poate fi vazut ca preview in coltul de dreapta jos a imaginii 3.1, unde deasemenea putem vedea cu o linie gri subtire marginea view-ului), cat si culoarea din background. Pentru view-ul prezentat de camera nu avem o dimensiune standard, deoarece o schimbam depinzand de cantitatea de mancare si de cate creaturi vrem sa folosim in cadrul unei simularii, insa pastram un aspect de imagine constant de 16:9. Obiectul "Walls" este compus din 4 pereti(reprezentati ca segmente albe in cadrul imaginii 3.1): **WallUp**, **WallRight**, **WallDown** si **WallLeft**. Acestia sunt obiecte statice, care nu se misca in timpul simularilor. Dimensiunea lor deasemenea nu este standard, dar la scalare menținem acelasi aspect de imagine precum camera. Fiecare dintre pereti are atasat un **Box Collider 2D** care este setat ca fiind Trigger, pentru a detecta coliziunile cu alte obieicte, si un **Rigidbody 2D** static plus

tag-ul ”Wall”, pentru a putea fi detectati prin intermediul **Raycast**(detalii in sectiunea 3.3). Atunci cand scalam peretii pentru diferite experimente o facem astfel incat marginile interioare ale peretilor sa fie inafara marginilor view-ului camerei, deoarece la generarea cu coordonate aleatorii ale mancarii si ale creaturilor, aceastea sunt generate intre marginile game view-ului, iar peretii fiind inafara lui evitam generarea obiectelor in pereti.

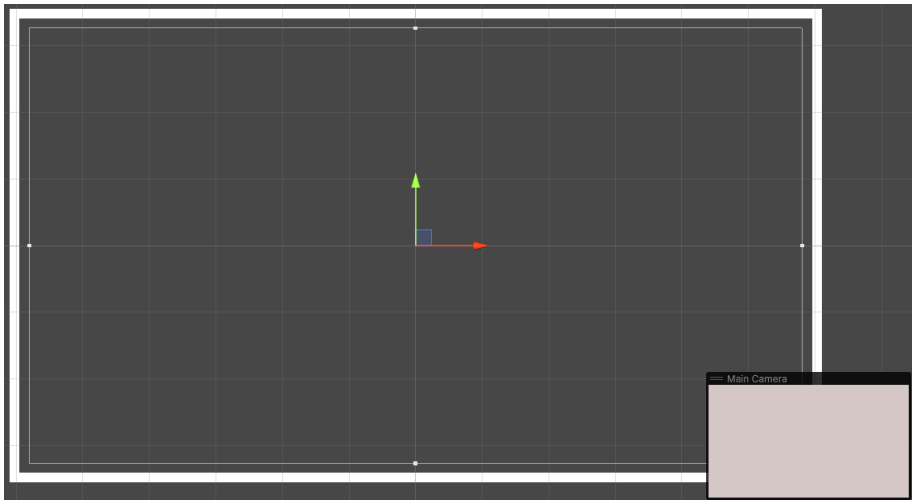


Figura 3.1: Main camera view din Unity

3.2 Mâncarea

Obiectele din ciclul de generare a mancarii sunt 2, **Food (Prefab asset)**(componente in imaginea 3.2) si **FoodManager**. **Food Prefab** reprezinta schema dupa care se construieste o bucata de energie. Bucatile de energie se gasesc pe harta sub forma de discuri de culoare galbena, mai exact #F6F800. Acestea, ca si peretii, reprezint obiecte statice care odata create nu se misca. **Food Prefab** are asociate urmatoarele componente:

- **Circle Colider 2D** permite detectarea coliziunilor cu Obiectul
- **Rigidbody 2D** permite ca obiectul sa poata fi lovit de Raycasts
- **Food Controller** un script C# care controleaza distrugerea obiectului.

Obiectele construite prin FoodPrefab au scala de 1 pe axa X, Y si Z, si sunt asociate cu tag-ul ”Food” pentru a permite detectia lor prin Raycasts si coleziuni. De asemenea,

ele sunt asociate cu **”Layer”**-ul ”Food” pentru a permite detectia lor de catre senzorul ”Field of View”(sectiunea 3.3).

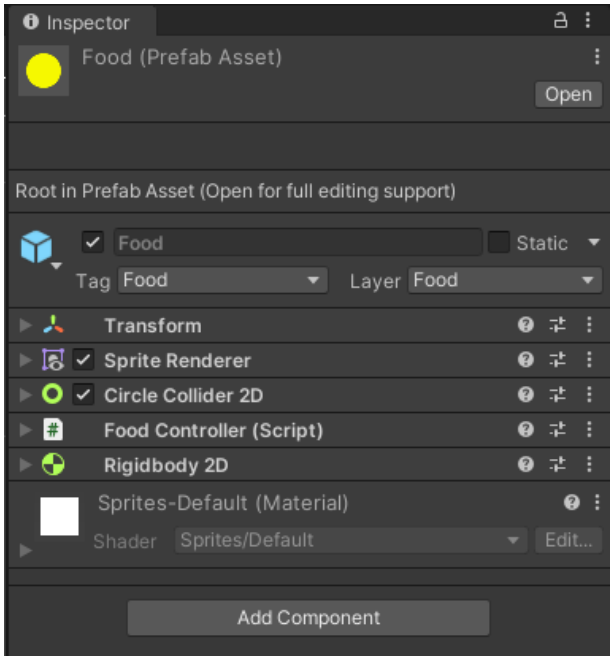


Figura 3.2: Componentele Prefab-ului Food

FoodManager este un ”Empty Object”, al carui singur scop este sa fie un obiect permanent in aplicatie de care putem atasa un script. Scriptul pentru **FoodManager** ne permite sa asociem acestuia un ”Prefab”, in cazul acesta **Food Prefab**, iar scriptul ne ofera functii pentru a crea un obiect sau mai multe din ”Prefab-ul” asociat, si ne permite si sa le distrugem pe toate. In **Food Controller**, scriptul pentru **FoodPrefab**, functia ”DieAndRespawn” permite distrugerea obiectului Food si regenerarea sa in alta locatie in mediu, printr-un apel la FoodManger. Astfel putem pastra numarul de bucati de energie constant in mediu, chiar daca energia este colectata in continuu de speciile de prada.

3.3 Creaturile

In mediul nostru virtual avem doua tipuri de creaturi, **Prada** si **Pradator**, care pot fi construite prin intermediul **Prey (Prefab Asset)** si respectiv **Predator (Prefab Asset)**. In aceasta vom enunta in mare parte proprietatile pentru **Prey Prefab**, deoarece cele 2 prefaburi au multe elemente comune, deci pentru **Predator Prefab** vom expune doar diferentele.

Prefab-ul pentru Prey defineste urmatoarele componente:

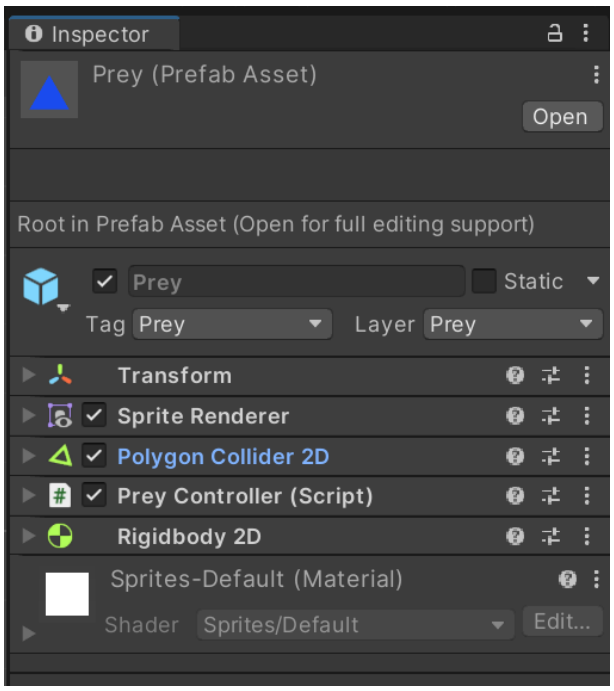


Figura 3.3: Componentele Prefab-ului Prey

- **Sprite Renderer**: Definim forma creaturilor prey ca fiind un triunghi, scalat la dimensiunea de 1.5 pe axa x si Y
- **Polygon Collider 2D**: Un Collider care ia forma sprite-ului, folosit pentru detectarea coleziunilor
- **Rigidbody 2D**: Necesar pentru detectarea coleziunilor, pentru ca detectarea coleziuniulor se va face in cadrul obiectelor dinamice: Prey si Predator. Este de tip ”Kinematic”, spre deosebire de static pentru obiectele statice, si permite detectarea coleziunilor kineamtic/static si kinematic/kinematic.
- **Prey Controller**: script C#, mosteneste clasa **Creature Controller**, care incapsuleaza trasaturi comune atat pentru prey cat si pentru predator. Defineste miscarea, senzorii, acumularea de energie si moartea pentru obiecte Prey.

Miscarea

Micarea este definita de schimbari care au loc asupra vitezei si a unghiului de rotatie al creaturii. O creatura se poate deplasa doar in fata sau in spate, dar aplicand rotatii asupra proprietatii ”transform.eulerAngles” se poate schimba directia in care aceasta merge. Schimbarile asupra vitezei si a rotatiei obiectului sunt determinate de o retea neuronala asociata obiectului, care primeste ca input un numar de valori

senzor, si returneaza doua valori in intervalul (-1, 1), prima fiind modificarea asupra acceleratiei iar a doua modificarea asupra rotatiei. Acest proces are loc in functia "FixedUpdate()", deci la fiecare schimbare de frame stratul de intrare se incarca cu valorile senzorilor, se executa pasul de feed-forward iar valorile din stratul de iesire sunt folosite pentru a modifica pozitia creaturii.

Senzori

In mod implicit, toate creaturile au urmatorii senzori:

- **Bias**: este un senzor care are mereu valoarea 1, folosit pentru a adauga bias la nodurile care creaza o conexiune cu el.

- **Perete**: acesta este un singur senzor, definit de un Raycast in fata obiectului(in planul 2D, aceasta reprezinta directia "up" pentru proprietatea "transform" a obiectului). Un raycast reprezinta proiectarea unei raze invizibile de o anumita lungime dintr-un punct, pe o directie in scena. Cand aceasta loveste collider-ul unui alt obiect din scena, este returnat un obiect din clasa "RaycastHit2D"(pentru mediul nostru 2D), care poate fi analizat pentru a determina ce tip de obiect a fost lovit(prin intermediul tag-ului) obiectului, si distanta de la punctul de pornire la care a fost lovit. Pentru senzorul de pereti, daca raycast-ul loveste un perete, valoarea sa va fi raportul dintre distanta pana la lovitura si lungimea razei(deci, intre 0 si 1). Daca nu a fost detectat un perete, valoarea senzorului e 1.

- **Viteza curenta**: valoarea senzorului este valoarea precedenta a iesirii pentru "viteza" a retelei neuronale.

- **Unghi**: unghiul curent, exprimat intre -1 si 1, dintre directia inainte a creaturii si axa Y a scenei.

Pentru detectia tintelor creaturilor(adica a surselor de energie pentru fiecare tip in parte, pradatorii detecteaza creaturi de tipul prada, iar prada detecteaza bucatile de energie), am folosit in timp 2 tipuri de senzori. Cel mai simplu reprezinta un sistem format din 3 Raycast-uri, o sa il numim **fork** pentru a ne referi la el mai departe, care actioneaza la fel ca senzorul implicit pentru perete, doar ca in loc sa detectez pereti, detecteaza tintele creaturii. Un raycast este proiectat pe directia inainte a obiectului, iar celelalte doua la un unghi de 45° respectiv -45° fata de aceasi directie.

Ulterior am dezvoltat un senzor mai complex, de tip "Field of View", al carui concept vine din [SBM05], sectiunea 5, figure 10, unde sunt descrisi mai multi senzori printre care si "Enemy Radars". Acesta descrie divizarea spatiului de $360°$ din jurul agentului in mai multe zone. Fiecare zona activeaza un senzor in dependenta de cat de apropiat este un inamic de agent. Daca sunt mai multi inamici in aceasi zona, se sumeaza activarile pentru fiecare inamic in parte.

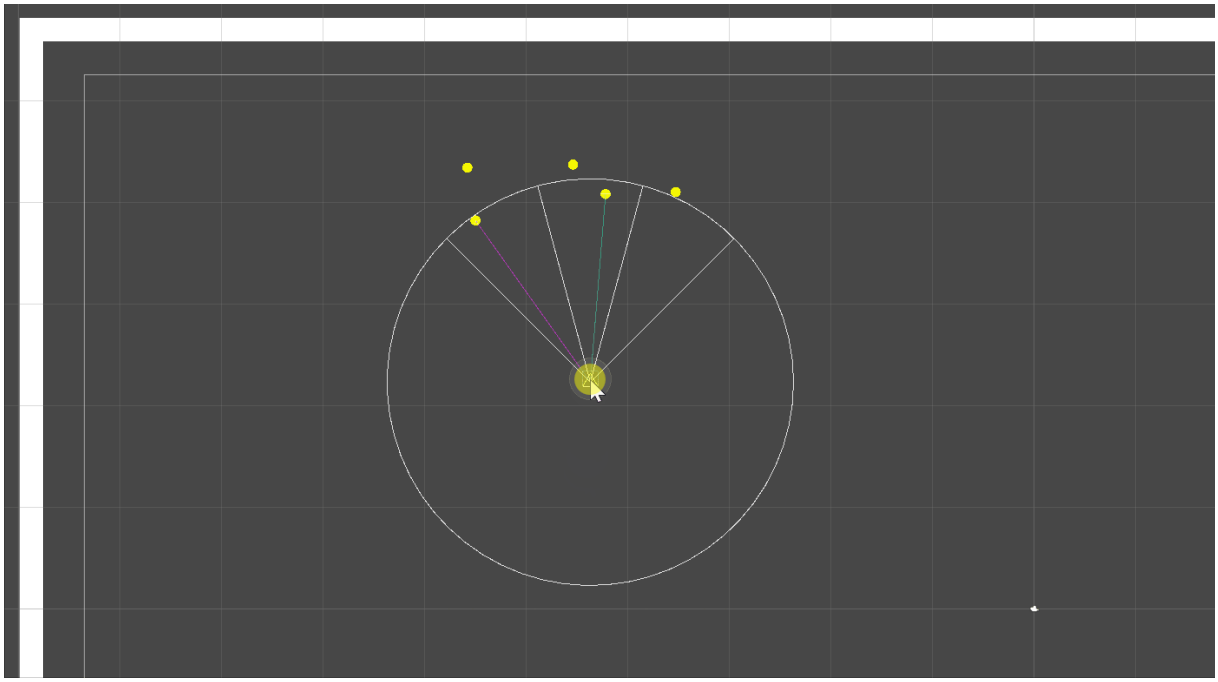


Figura 3.4: Senzorul FoV si senzorul pentru perete din cadrul prazii, liniile colorate arata distnata pana la fragmentele de energie care sunt detectate de diferitele zone ale senzorului. Linia alba atunci cand prada se apropie de perete este raycast-ul catre perete

In cadrul implementarii noastre, pentru senzorul "Field of View"(**FoV**), alegem un unghi, intre 0 si 180 de grade, care reprezinta campul vizual din fata unei creaturi, si raza pana la care acesta se extinde. Prin alt parametru, specificam in cate zone se imparte acesta. Prin intermediul metodei "Physics2D.OverlapCircleAll", se detecteaza toate tintele din cadrul unui cerc din jurul creaturii, dupa care, analizand unghiul dintre directia inainte a creaturii si pozitia unei tinte, putem determina daca tintele cad in cadrul campului vizual, si in ce zona a campului. Pentru fiecare zona, activarea ei va reprezenta valoarea calculului $\sum_i 1 - d_i/r$, i fiind o tinta din cadrul zonei, d_i distanta pana la tinta, iar r raza cercului in care are loc detectia. Deci pentru fiecare tinta avem o activarea intre 0 si 1, care este mai mare cu cat tinta e mai aproape de creatura. In

cadrul animatiei 3.4, se poate vedea cum functioneaza senzorul ”FoW”, cu un camp vizual de $90°$, impartit in 3 zone, pe o distanta de 20 ”world units”, iar la sfarsitul animatie putem vedea si o vizualizare a raycast-ului pentru senzorul de perete. Pentru a scrie vizulizare in ”Editor View” a senzorului ”FoW” un mare ajutor a fost [Lag15]. In unul din experimentele de la capitolul 6, vom arata ca prada care foloseste senzorul ”FoW” este superioara celei care foloseste ”Fork”, si vom prezenta si diferentele de comportament intre cele 2 tipur de prada. Pentru pradatori la senzori nu se schimba nimic decat tintele cautate, acestea fiind creaturile de prada.

Coleziuni

Datorita faptului ca toate obiectele care intereactioneaza in cadrul scenei au asociat cate un collider si un rigidbody, putem detecta coleziunile intre ele. Procesare acestor coleziuni se face in cadrul script-uri-lor ”Prey Controller” si ”Predator Controller”, asociate prefab-ului ”Prey” si respectiv ”Predator”. Prin intermediul tag-urilor ”Wall” si ”Food”, script-ul ”Prey Controller” inregistreaza coleziunile cu peretii, ceea ce cauzeaza moartea prazii, si coleziunea cu fragmentele de energie, prin care energia asociata fragmentului este absorbita, iar fragmentul prin functia ”DieAndRespawn” este regenerat la alte coordonate aleatorii in cadrul scenei, deci cantitatea de energie in scena este constanta. Procesare coleziunilor prada-pradator are loc in script-ul ”Predator Controller”, si pot fi identificate prin tag-ul asociat creaturilor de prada, ”Prey”. In cadrul acestor coleziuni, obiectul prada este distrus, insa obiectul pradator primeste energie, intr-o cantitate semnificativ mai mare decat ce primesc creaturile prada la coleziunile prada-fragment de energie, plus un procent din energia acumulata de prada de-a lungul vietii. Facem aceste ajustari deoarece vanatul pentru prada este mult mai usor decat cel pentru pradator,pentru ca pradatorii trebuie sa dezvolte comportamente de chassing, astfel ca prdatorii ar trebui sa primeasca mai multa energie din vanat pentru a ne asigura ca acestia nu sunt surclasati de prada in cadrul evolutiei. Coleziunile pradator-perete sunt la fel ca la prada, insa pradatorii nu inregistreaza coleziunile cu fragmentele de energie, deoarece vrem ca acestia sa obtina energie doar prin coleziunile cu prada. Pe tot parcursul vietii lor, creaturile consuma energie in mod constant, plus o cantitate care depinde de viteza cu care se misca. Astfel le incurajam sa acumuleze energie incontinuu. Odata ce o creatura ramane fara energie, aceasta moare.

3.4 Managementul Algoritmului

NeatManager, ca si FoodManger, este un "Empty Object" pe care il folosim pentru a atasa script-ul "Neat Manager". Acest script este responsabil de desfasurare algoritmului Neat, deci are ca responsabilitati generarea populatiei initiale, inregistrarea mortilor, crearea generatiilor urmatoare si inregistrarea de log-uri despre populatie pe parcursul generatiilor. In cadrul NeatManager putem decide sa efectuam experimente in care folosim doar creaturile de prada, sau si pradatorii, si putem determina paramteri cum ar fi numarul de membri, numarul de noduri pentru stratul de intrare si cel de iesire etc.(vom discuta mai pe larg multimea de parametri in sectiunea 5). Faza de generare a populatiiei initiale se desfasoara in urmatorul mod:

1. Se genereaza genomul initial pentru fiecare membru din populatie(reprezinta o codificare Neat a unei retele cu structura minima dar cu ponderi ale conexiunilor aleatorii, revedeti sectiunea 1.2.2)

2. Se genereaza fenotipul pentru fiecare genotip(reteaua neuronala propriu-zisa, vom expune maparea de la fenotip la genotip in capitolul 4)

3. Se creaza agentii simularii, dupa prefab-ul care a fost asignat catre NeatManager. Proprietatile acestora sunt setate dupa parametrii NeatManager pentru un prefab specific, si sunt pozitionati la coordonate aleatorii in cadrul "Main Camera View", fiecaruia dintre ei fiindui asignata si una dintre retele create la pasul precedent.

Dupa creare, agentii sunt liberi sa exploreze mediul si sa acumuleze fitness. In sectiunea anterioara nu am mentionat ca in toate scenariile in care un agent moare, asta fiind valabil atat pentru prada cat si pentru pradatori, acesta isi inregistreaza moartea si cauza ei la "NeatManager", prin intermediul functiei "RegisterDeath". Astfel, NeatManager poate determina cand toti agentii din mediu au fost distrusi, si poate incepe crearea urmatoarei generatii. Inainte de crearea urmatoarei generatii, se log-eaza fitness-ul average si fitness-ul celui mai bun agent din generatia curenta, plus informatii despre structura celui mai bun agent. Procesul de generare a urmatoarei generatii decurge la fel ca cel de initializre, doar ca mai intai se aplica o filtrare pentru a ramane doar genom-urile cele mai bune din cadrul fiecarei specii, care vor trece prin procesul de crossover si de mutatie pentru generarea genomurilor generatiiei viitoare. Aceste genom-uri sunt asignate catre diferite specii din populatie, care grupeaza genomuri similare. Dupa acest pas se creaza noile retele neuonale, si se genereaza noii agenti care

gandesc prin intermediul acestora, iar tot procesul se repeta din nou la fiecare generatie. Ne-am permis in aceasta sectiune sa facem un rezumat al functionarii algoritmului, vorbind despre obiectul care este responsabil de desfasurarea lui, insa in capitolul 4 vom expune detaliat, cu exemple din cod, procesele din cadrul algoritmului.

Capitolul 4

Detalii de implementare

In cadrul algoritmului NEAT, expus in [SM02] sunt expuse in mod teoretic inovatiile aduse de NEAT ca algoritm de Neuro-Evolutie, insa nu avem o implementare a lor, cum ar fi, de exemplu, generarea numerelor de inovatie. In acest capitol vom concretiza aspecte putin expuse in NEAT, aspecte care tin doar de de simulare, precum si imbunatatiri sau schimbari propuse mai tarziu pe "The NEAT Users Page"[Sta15], in sectiunea "Neat Software FAQ".

4.1 Genomul in NEAT

Structura genomului in NEAT este foarte simpla, si intuitiv de implementat, la fel si operatiile din cadrul mutatiei, care implica perturbarea ponderilor conexiunilor, resetare ponderilor cu noi valori aleatorii, adaugarea de noi noduri si adaugarea de conexiuni, descrise bine in "Figure 3" din [SM02]. In cadrul operatiei de crossover, deasemenea, prin intermediul fitnessului este usor sa intelegem genele carui parinte vor fi mostenite de copil. Totusi, pentru operatia de aliniere a genom-urilor parintilor, exemplul din "Figure 4" din [SM02] nu este destul de bun, deoarece el ascunde complexitatile operatiei in cazul in care genele matching nu sunt toate la inceput-ul fiecaruia dintre parinti, ci se pot gasi si printe genele "disjoint".

Listing 4.1: Parcurgerea genom-urilor celor 2 parinti pentru alinierea lor in cadrul crossover. Prin "this" sunt accesate campurile primului parinte

```
1    int idx1 = 0, idx2 = 0;
2    while (idx1 < this.connectionGenes.Count && idx2 < parent2.
         connectionGenes.Count)
3    {
```

```
        var p1Gene = this.connectionGenes[idx1];
        var p2Gene = parent2.connectionGenes[idx2];
        if (p1Gene.inovationNr == p2Gene.inovationNr)
        {
            child.AddMatchingGene(p1Gene, p2Gene, inheritDisabledGeneProb);
            idx1++;
            idx2++;
        }
        else if (p1Gene.inovationNr < p2Gene.inovationNr)
        {
            if (p1Fitness >= p2Fitness)
            {
                child.AddCopyOfGene(p1Gene);
            }
            idx1++;
        }
        else
        {
            if (p2Fitness > p1Fitness)
            {
                child.AddCopyOfGene(p2Gene);
            }
            idx2++;
        }
    }

    if (idx1 < this.connectionGenes.Count && p1Fitness >= p2Fitness)
    {
        for (int i = idx1; i < this.connectionGenes.Count; i++)
        {
            child.AddCopyOfGene(this.connectionGenes[i]);
        }
    }
    else if (idx2 < parent2.connectionGenes.Count && p2Fitness > p1Fitness)
    {
        for (int i = idx2; i < parent2.connectionGenes.Count; i++)
        {
            child.AddCopyOfGene(parent2.connectionGenes[i]);
        }
    }
```

Solutia noastra, expusa in 4.1, propune folosirea a 2 indexi pentru parcurgerea ambelor genomuri in acelasi timp. Daca pentru pozitia curenta a indexilor ambele gene au numere de inovatie egale, avansam ambii indexi, daca parintele 1 are numarul de inovatie mai mare, avansam indexul parintelui 2, deoarece este posbil ca la urmatoare pozitie genele sa fie matching, iar altfel, avansam index-ul pentru parintele 1. Daca ajungem la sfarsit si genomul unui parinte sa terminat inaintea celuilat, inseamna ca restul genelor din parintele care nu sa epuizat sunt exces. Aceasi aliniere o facem si in cadrul operatiei de calcul al distante δ dintre 2 genom-uri, doar ca in loc sa asignam gena unui sau a altui parinte la copilul ce se formeaza, contorizam genele ”matching”, cele disjoint si cele ”excess”.

4.2 Retelele neuronale din NEAT

Retelel neuronale din NEAT reprezinta retele neuronale recurente. Acestea au cicluri care pot porni din straturile adanci ale retelei spre cele din fata, cum ar fi de la cele de intrare la cele de iesire, si nu este garantat ca fiecare nod dintr-un strat va fi conectat la fiecare nod din urmatorul, deci reprezentarea si evaluarea din cadrul retelelor dense feed-forward nu poate fi aplicata aici. Lucrarea [SM02] nu are specificatii pentru generarea retelelor, inafara de faptul ca genele de conexiune setate ca disabled nu ar trebui sa fie exprimate in fenotip, adica in retea.

Listing 4.2: Structura pe clase a unei retele NEAT

```
public class NeatNetwork
{
    public float fitness;
    public int index;
    private enum CycleDetection { Unverified, Verified, Cycle }
    private readonly List<int> evalOrder;
    private  List<int> inputs;
    private  List<int> outputs;
    private readonly Dictionary<int, Node> nodes;
    ...
}

internal class Node
{
    public int id;
```

```
    public float value;
    public List<Connection> inputConnections;
    public List<Connection> outputConnections;
    ...
}

internal class Connection
{
    public int inNode;
    public int outNode;
    public float weight;
    ...
}
```

O retea, dupa cum am listat in 4.2, are urmatoarele componente:

- **fitness**: stocheaza fitness-ul unui agent din populatie odata cu moartea sa.
- **index**: un id al retelei asignat de catre NeatManager.
- **nodes**: un dictionar constituit din perechi de id-uri ale nodurilor si noduri. Contine toate nodurile necesare retelei, iar fiecare nod are propriile liste de conexiuni cu alte noduri.
- **evalOrder**: o lista de id-uri de noduri, reprezentand ordinea in care nodurile ar trebui evaluate in retea pentru a obtine un output corect.
- **inputs si outputs**: liste cu id-urile nodurilor de intrare, respectiv de iesire.
- **CycleDetection**: un enum ce va fi folosit pentru stabilirea conexiunilor care formeaza un ciclu in retea.

Prin stabilirea unei ordini de evaluare a nodurilor din retea la crearea retelei, evaluarea va putea avea loc simplu si rapid, ceea ce e importanta deoarece o retea va fi evaluata de foarte multe ori pe parcursul vietii sale, la fiecare frame al simularii. Pentru generare unei retele RNN din genotip, vom trece prin 4 pasi:

1. Filtrarea conexiunilor enabled si crearea nodurilor necesare pentru conexiuni.
2. Detectarea ciclurilor din cadrul retelei, ne referim la conexiunile care completeaza un ciclu.

3. Eliminarea structurilor care nu sunt conectate la nici un nod de intrare, acestea nu primesc date de la senzori, deci sunt redundante si nu aduc nici un beneficiu retelei.

4. Stabilire unei ordini de evaluare a nodurilor, cu ajutorul informatiei despre cicluri pe care o avem de la pasul 2.

Pentru prima rutina, intii cream nodurile care corespund straturilor de intrare si celor de iesire pe care le adaugam in `nodes`. Ulterior, pentru fiecare "connection gene" care nu este dezactivata, verificam daca nodul de intrare si cel de iesire sunt prezente in `nodes`, altfel, le cream si le adaugam. Prin informatia din "connection gene", putem construi un nou obiect "Connection", iar aceasta conexiune o adaugam in lista de `outputConnections` pentru nodul de intrare, si cea de `inputConnections` pentru nodul de iesire. Astfel, fiecare nod stie cu ce alte noduri este conectat, ponderea conexiunilor si directia conexiunilor. Pastram si o lista a tuturor conexiunilor create care va fi folosita la pasul urmator.
In pasul 2, pentru determinarea nodurilor ce formeaza cicluri, folosim algoritmul 4.3.

Listing 4.3: Algoritmul de detectare a conexiunilor ce formeaza cicluri

```
1  private Dictionary<Connection, (CycleDetection verification, bool mark)>
       DetectCycles(List<Connection> connections)
2  {
3      Dictionary<Connection, (CycleDetection verification, bool mark)>
           verificationDictionary = new();
4      connections.ForEach(conn => verificationDictionary.Add(conn, (
           CycleDetection.Unverified, false)));
5      foreach (var nodeId in inputs)
6      {
7          var currRoute = new List<int>() { nodeId };
8          ExploreRouteCiclicity(currRoute, verificationDictionary);
9      }
10
11     return verificationDictionary;
12 }
13
14 private void ExploreRouteCiclicity(List<int> route, Dictionary<Connection,
       (CycleDetection verification, bool mark)> verificationDictionary)
15 {
16     foreach (var conn in nodes[route[^1]].outputConnections)
```

```
17      {
18          if (verificationDictionary[conn].verification == CycleDetection.
                Unverified)
19          {
20              if (route.Contains(conn.outNode))
21              {
22                  verificationDictionary[conn] = (CycleDetection.Cycle, false
                        );
23              }
24              else
25              {
26                  verificationDictionary[conn] = (CycleDetection.Verified,
                        false);
27                  ExploreRouteCiclicity(new List<int>(route) { conn.outNode }
                        , verificationDictionary);
28              }
29          }
30      }
31  }
```

Ideea este sa initializam un dictionar unde cheile sunt conexiunile, deoarece ele sunt unice, iar valorile sunt statusul verificarii conexiunii(initial toate conexiunile sunt marcate ca fiind unverified), plus un marcaj ce va fi folosit la pasul 4. Dupa care, pentru fiecare nod de intrare, incepem o cautare "Depth-first search"(DFS). Practic exploram toate conexiunile de iesire a nodului curent, iar pentru conexiunile care sunt marcate ca neverificate, daca nodul de iesire al conexiunii este deja prezent in ruta pe care am parcurso pentru a ajunge la nodul curent, insemna ca am gasit un ciclu, si marcam conexiunea in acest sens, altfel, marcam conexiunea ca fiind verificata, iar nodul de iesire al conexiunii este adaugat in ruta ca fiind nodul curent si continuam explorarea. Totusi, daca analizam un exemplu de retea cum ar fi cel din imaginea 4.1, vedem ca nu toate conexiunile sunt verificate. Verificarile incep de la fiecare nod din stratul de intrare in parte, deci o conexiune care ramane neverificata este una la care nu ajung prin nici o cale informatiile din stratul de intrare. Astfel, structurile neconectate pot fi sterse fara sa afecteze in nici un mod evaluarea retelei, ci chiar faciliteaza procesul de stabilire a ordinii de evaluare a nodurilor. Procesul de eliminare al structurilor redundnate este simplu: fiecare conexiune ramasa in starea unverified va fi stearsa din dictionarul de verificari, si din listele nodurilor pe care le conecteaza, dupa care este

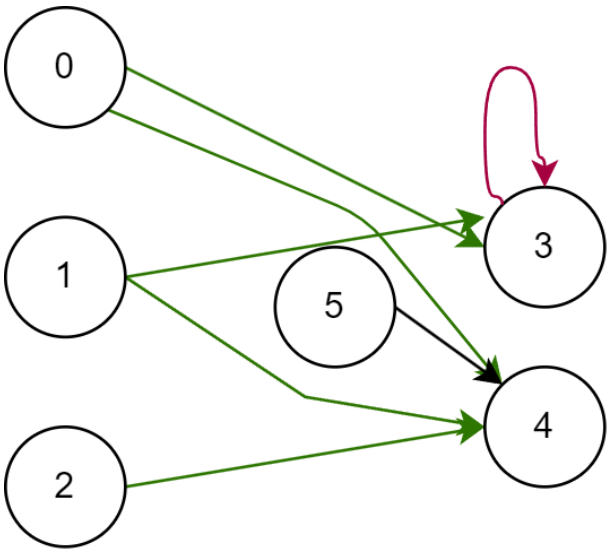


Figura 4.1: Un exmplu de retea dupa verificarea ciclicitatii conexiunilor. Sagetile verzi sunt conexiunile cu starea de verified, cele rosii au starea de cycle, insa putem vedea ca conexiunea 5-4 ramane unverified(sageata neagra), datorita faptului ca nu este conectata la nici un input.

parcurs dictionarul `nodes` si sunt eliminate toate nodurile ascunse care nu au nici o conexiune de input. Ultima parte, stabilirea ordinii de evaluare a nodurilor este expusa in listing-ul 4.4. Logica acestei operatii este urmatoare: un nod poate fi evaluat odata ce au fost evaluate toate nodurile cu care el este conectat prin lista de conexiuni `inputConnections`, cu exceptia nodurilor de intrare ce sunt conectate de nodul curent prin conexiuni ciclu, deoarece de la aceste nodul curent va lua valoarea lor de la evaluarea anterioara. Astfel, la inceput, aceasta conditie este indeplinita doar de o parte din nodurile de intrare, care nu au `inputConnections` sau care au doar cicluri. Definim o lista, `currOrder` in care punem nodurile care indeplinesc aceasta conditie, iar intro bucla, pana cand toate nodurile sunt incluse in `evalOrder`:

1. Cu ajutorul `verificationDictionary`, pentru fiecare nod din `currOrder`, schimbam marcajul conexiunilor sale de iesire din $false$ in $true$.

2. Cream o lista goala next.

3. Pentru fiecare nod care a ramas sa fie asignat ordinii: daca toate conexiunile sale de intrare sunt cicluri sau sunt marcate true, asta insemnand ca nodul de intrare indicat de conexiune a fost deja adaugat in ordinea de evaluare, deci in timpul evaluarii valoarea sa ar fi deja calculata, nodul curent poate fi adaugat in lista next.

4. Adaugam la `evalOrder` nodurile din `currOrder`, iar noul `currOrder` va fi lista next.

Astfel, gradual, la fiecare iteratie a buclei, nodurile care pot fi evaluate cu informatia valabila la moment for fi adaugate in ordinea de evaluare, pana cand aceeasta este completata cu toate nodurile retelei.

Listing 4.4: Functia de determinare a ordinii de evaluare a nodurilor

```
private void EstablishEvalOrder(Dictionary<Connection, (CycleDetection
    verification, bool mark)> verificationDictionary)
{
    var remaining = nodes.Keys.ToList();

    var currOrder = inputs.Where(nodeId => nodes[nodeId].inputConnections.
        All(conn => verificationDictionary[conn].verification ==
        CycleDetection.Cycle)).ToList();
    foreach (var node in currOrder)
    {
        remaining.Remove(node);
    }

    while (remaining.Count > 0)
    {
        foreach (var nodeId in currOrder)
        {
            foreach (var con in nodes[nodeId].outputConnections)
            {
                if (verificationDictionary[con].verification ==
                    CycleDetection.Verified)
                {
                    verificationDictionary[con] = (CycleDetection.Verified,
                        true);
                }
            }
        }

        var next = new List<int>();
        for(int i = 0; i < remaining.Count; i++)
        {
            var nodeId = remaining[i];
            if (nodes[nodeId].inputConnections.All(conn =>
```

```
                verificationDictionary[conn].mark || verificationDictionary[
                conn].verification == CycleDetection.Cycle))
            {
                next.Add(nodeId);
                remaining.RemoveAt(i);
                i--;
            }
        }

        evalOrder.AddRange(currOrder);
        currOrder = next;
    }

    evalOrder.AddRange(currOrder);
}
```

4.3 Algoritmul NEAT

Responsabil de executia algoritmului NEAT in cadrul proiectului nostru este script-ul "NeatManager.cs" atasat obiectului NeatManager din scena. Cea mai mare parte a algortmului este integrate in cadrul functiei `ProduceNextGeneration()` (De vazut listing-ul 4.5)

Listing 4.5: Generarea urmatoarei generatii

```
private void ProduceNextGeneration()
{
    //inovationManger.ResetForNextGeneration();
    AdjustCompatibiltyTreshold();
    ComputeAllSpeciesAdjustedFitness();
    AdjustAllSpeciesPotentialMemeberCount();
    GenerateNextGenGenomes();
    SpeciesRepartition();
    GenerateNetworks();
    GenerateCreatures();
    currentGeneration++;
}
```

O sa observati ca prima linie este comentata: de ce? In cadrul implementarii noastre, `InovationManger` este o clasa a carei obiecte ne permit sa tinem cont de

inovatiile ce apar in genom-urile populatie in urma mutatiilor structurale. In pagina [Sta15], este adresata intrebarea daca ar trebuie sa pastram inregistraile noilor inovatii doar pentru generatia curenta sau pe tot parcursul algoritmului. Ca raspuns Keneth O. Stanley precizeaza ca efectul principal al pastrarii numerelor de inovatie pe toat parcursul algoritmului este ca se vor genera mai putine specii, dar nu a definit o metoda ca fiind mai buna ca alta, asta depinzand de experiment. In cadrul simulatorului nostru totusi, pastrarea permanenta a numerelor de inovatie a produs rezultate mai bune, din acest motiv linia 3 este comentata, pentru a nu reseta inregistrarile la fiecare generatie. Functia `AdjustCompatibiltyTreshold` deasemenea este inspirata din unul din raspunsurile de pe pagina [Sta15], anume daca exista o metoda pentru selecta in mod automat pragul de compatibilitate δ_t pentru speciatie. Aceasta metoda implica setare unui numar tinta de specii in cadrul algoritmului, iar acest pragul de compatibilitate va scadea daca avem mai putin specii decat numarul tinta, sau va creste dca sunt prea multe specii. In cadrul implementarii proprii am setat un minim si un maxim pentru acest prag, si o valoare prin care pragul se modifica la inceputul fiecarei generatii(vedeti 4.6).

Listing 4.6: Ajustarea pragului de compatibilitate

```
1  private void AdjustCompatibiltyTreshold()
2  {
3      if (currentGeneration > 1)
4      {
5          if (species.Count < speciesTargetCount && compatibilityTreshold >
               minCompatibilityTreshold)
6              compatibilityTreshold -= compatibilityModif;
7          else if (species.Count > speciesTargetCount &&
               compatibilityTreshold < maxCompatibilityTreshold)
8              compatibilityTreshold += compatibilityModif;
9
10         if (compatibilityTreshold < minCompatibilityTreshold)
               compatibilityTreshold = minCompatibilityTreshold;
11         if (compatibilityTreshold > maxCompatibilityTreshold)
               compatibilityTreshold = maxCompatibilityTreshold;
12     }
13 }
```

Fiecare membru din populatie apartine unei specii.Apartenent la o anumita specie e determinata de genomul membrului. O specie in cadrul implementarii noastre

are urmatorea structura:

Listing 4.7: Reprezentarea unei specii in algoritm

```
public class Species
{
    public int Id { get; set; }
    public List<int> MembersIdx { get; set; }
    public NeatGenome SpeciesRepresentative { get; set; }
    public int PotentialMembersCount { get; set; }
    public float LastBestFitness { get; set; }
    public int GenerationsSinceImprovement { get; set; }
    public int GenerationsSinceMemebersAssigned { get; set; }
    public float SpeciesAdjustedFitness { get; set; }
    public (byte r, byte g, byte b) RgbColor { get; set; }
    ...
}
```

Vom descrie aceste proprietati in cadrul metodeleor care le acceseaza.

In `ComputeAllSpeciesAdjustedFitness()` se intimpla urmatoarele operatii: pentru fiecare specie, membrii din `MembersIdx` sunt sortati dupa fitness-ul retelelor lor, se determina daca exista o imbunatatire a celei mai bun fitness al speciei de pana acum accesand `LastBestFitness`, iar in caz contrar se incrementeaza counterul `GenerationsSinceImprovement`, ceea ce duce la eliminarea speciei daca aceasta a stagnat de prea multe generatii. Dupa care se calculeaza media fitness-ului membrilor speciei, ce este asignata proprietatii `SpeciesAdjustedFitness`.
Aceast medie va fi folosit in urmatoarea functie, `AdjustAllSpeciesPotentialMemeberCoun`
unde se calculeaza, ca un raport intre fitness-ul mediu al speciei si suma fitness-urilor medii ale tuturor speciilor ce procent din populatie vor fi potentiali membri ai speciei, asa se calculeaza numarul de membri potentiali, `PotentialMembersCount`. Tot in cadrul acestei functii eliminam si membrii speciei curente care au performante mai proaste. De exemplu daca pastram doar 30% dintre mebri sunt eliminati ceilalti 70%
Numarul de membri potentiali pentru fiecare specie este folosit in metoda `AdjustAllSpeciesPo`
pentru a determina de cate ori vom utiliza o specie pentru alegerea in mod aleator a parintilor unui nou copil, a carui genom e generat prin crossover si mutatie in cadrul metodei `GenerateNextGenGenomes`. Tot in aceasta metoda pentru fiecare specie existenta va fi ales in mod aleator dintre membrii sai un reprezentant, inainte initierii procesului de creare de noi copii care vor inlocui populatia curenta. De remarcat ca

operatorii genetici pot aduce schimbari asupra genomului copilului astfel incat acesta nu se mai incadreaza in nici una din speciile parintilor sai(in cazul in care are loc crossover intre 2 indivizi din specii diferite), insa adoptand o sugestie din cadrul [Sta15] pentru a mari viteza cu care are loc specierea, copilul va primi ca `speciesHint` proprietatea `Id` a speciei primului parinte.

Metoda `SpeciesRepartition()` se ocupa de repartizarea genom-urilor noi create la pasul anterior la cate o specie. Pentru fiecare genom care reprezinta noua populatie, determinam prin intermediul proprietatii sale `speciesHint`, daca acesta apartine la aceasi specie ca parintele său, calculand distanta de compatibilitate δ dintre `SpeciesRepresentative` si genom-ul curent. Daca δ nu este mai mic decat pragul de compatibilitate, adica genomul copilului nu apartine speciei parintelui, refacem acelasi calcul cu fiecare dintre celelalte specii. Daca genomul curent nu este compatibil cu niciuna dintre speciile din populatie, este creata o noua specie, care primeste un nou `Id` unic, o culoare pe care o generam pe baza id-ului(aceasta va fi culoarea obiectelor din scena care reprezinta membri ai speciei), reprezentantul speciei va fi genomul curent, si tot el va fi si unicul membru, daca nu sunt asignati si alte genom-uri acestei specii.

In cadrul acestei metode vom executa si operatia de eliminare a speciilor redundante. Acestea sunt specii care nu au avut asignat nici un membru timp de mai multe generatii, valoare care o putem accesa prin prorietatea `GenerationsSinceMemebersAssigned` a speciei. Nu eliminam specia inca de la prima generatie in care nu i-a fost asociat nici un membru, deoarece e posibil ca in timpul unei generatii ulterioare aceasteia sa ii fie asociat unul, deci evitam crearea de specii noi fara motiv.

Toate genom-urile populatiei noi au fost asociate speciilor cu care sunt compatibile, asa ca tot ce a mai ramas e ca pentru fiecare individ din populatie sa ii generam din genom reteau neuronala dupa cum am descris in sectiunea 4.2, prin metoda `GenerateNetworks()`, si sa generam creaturile din scena care vor folosi aceste retele prin metoda `GenerateCreatures()` pe care o expunem in listing-ul 4.8 ca un exemplu pentru cum arata generarea unei populatii de prada, cu tot cu parametri.

Listing 4.8: Generarea speciilor de prada

```
1 var foodMask = LayerMask.GetMask("Food");
2 foreach (var s in species)
3 {
4     foreach (int member in s.MembersIdx)
```

```
    {
        var pos = new Vector2(UnityEngine.Random.value, UnityEngine.Random.
            value);
        pos = Camera.main.ViewportToWorldPoint(pos);

        allCreatures[member] = Instantiate(creaturePrefab, pos, transform.
            rotation);
        var creatureController = allCreatures[member].gameObject.
            GetComponent<PreyController>();
        currentPopulation++;

        creatureController.network = allNetworks[member];
        creatureController.inputNodes = inputNodes;
        creatureController.outputNodes = outputNodes;
        creatureController.totalEnergy = startingEnergy;
        creatureController.energyForFood = energyForFood;

        creatureController.fowTargetsMask = foodMask;
        creatureController.wallRayDistance = 20f;
        creatureController.viewDistance = 20f;
        creatureController.viewAngle = 90f;
        creatureController.angleSplits = 3;
        (var r, var g, var b) = s.RgbColor;
        creatureController.spriteColor = new Color32(r, g, b, 255);
    }
}
```

Capitolul 5

Setari experimentale

Capitolul 6

Rezultate

Concluzii

Bibliografie

[Rec73] Ingo Rechenberg, *Evolutionsstrategie; Optimierung technischer Systeme nach Prinzipien der biologischen Evolution. Mit einem Nachwort von Manfred Eigen*, Frommann-Holzboog, 1973.

[DM92] D. Dasgupta și D. McGregor, „Designing application-specific neural networks using the structured genetic algorithm", în *Proceedings of the International Conference on Combinations of Genetic Algorithms and Neural Networks*, ed. de D. Whitley și J. D. Schaffer, IEE Press, 1992, pp. 87–96.

[Hol92] John H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*, Cambridge, MA, USA: MIT Press, 1992, ISBN: 0262082136.

[Gru93] Frederic Gruau, „Genetic Synthesis of Modular Neural Networks", în *Proceedings of the 5th International Conference on Genetic Algorithms, ICGA-93*, ed. de Stephanie Forrest, University of Illinois at Urbana-Champaign: Morgan Kaufmann, Iul. 1993, pp. 318–325, ISBN: 1-55860-299-2, URL: http://dl.acm.org/citation.cfm?id=645513.657409.

[MM95] David Moriarty și Risto Miikkulainen, „Efficient Reinforcement Learning Through Symbiotic Evolution", în *Machine Learning* 22 (Ian. 1995), DOI: 10.1023/A:1018004120707.

[GM99] Faustino Gomez și Risto Miikkulainen, „Solving Non-Markovian Control Tasks with Neuro-Evolution.", în Ian. 1999, pp. 1356–1361.

[SM02] Kenneth O. Stanley și Risto Miikkulainen, „Evolving Neural Networks through Augmenting Topologies", în *Evolutionary Computation* 10.2 (2002), pp. 99–127, DOI: 10.1162/106365602320169811.

[Uni03] Department of Computer Sciences University of Texas at Austin, *Neuro Evolving Robotic Operatives*, June 16, 2023, 2003, URL: https://nn.cs.utexas.edu/nero/.

[YLH04] G.N. Yannakakis, J. Levine și J. Hallam, „An evolutionary approach for interactive computer games", în *Proceedings of the 2004 Congress on Evolutionary Computation (IEEE Cat. No.04TH8753)*, vol. 1, 2004, 986–993 Vol.1, DOI: 10.1109/CEC.2004.1330969.

[SBM05] K.O. Stanley, B.D. Bryant și R. Miikkulainen, „Real-time neuroevolution in the NERO video game", în *IEEE Transactions on Evolutionary Computation* 9.6 (2005), pp. 653–668, DOI: 10.1109/TEVC.2005.856210.

[Hor+06] Greg Hornby et al., „Automated Antenna Design with Evolutionary Algorithms", în *Collection of Technical Papers - Space 2006 Conference* 1 (Sept. 2006), DOI: 10.2514/6.2006-7242.

[Oro13] Daniel Oro, „Grand challenges in population dynamics", în *Frontiers in Ecology and Evolution* 1 (2013), ISSN: 2296-701X, DOI: 10.3389/fevo.2013.00002, URL: https://www.frontiersin.org/articles/10.3389/fevo.2013.00002.

[Lag15] Sebastian Lague, *Field of view visualisation (E01)*, Youtube, 2015, URL: https://www.youtube.com/watch?v=rQG9aUWarwE&list=PLFt_AvWsXl0dohbtVgHUY7xZv7.

[Sta15] Kenneth O. Stanley, *The NeuroEvolution of Augmenting Topologies (NEAT) Users Page*, University of Central Florida - College of Engineering și Computer Science, 2015, URL: http://www.cs.ucf.edu/~kstanley/neat.html.

[Gel16] Sanchez M. Gelles T., „Comparing Adaptivity of Ants using NEAT and rt-NEAT", în (2016).

[DeL20] Lyon S. DeLong JP, „Temperature alters the shape of predator-prey cycles through effects on underlying mechanisms.", în *PeerJ* (2020), DOI: 10.7717/peerj.9377.

[Zol21] Zolphite, *Simulated Fish Ecosystem (Neural Networks from Scratch + NEAT Genetic Algorithm) Code Along in Unity*, Youtube, 2021, URL: https://www.youtube.com/watch?v=0O2AUKQPAFc&list=PLC8R0n_dfXXr0fc4NdsInilobpp=iAQB.